

Java Collections Framework

List · Set · Map · Queue · Iterator · Algorithms

Faculty of Information Technologies
Industrial University of Ho Chi Minh City

Chapter Contents

01 Why Collections?

Arrays vs Collections · Framework · Demo

02 Collection Hierarchy

Interfaces · Architecture · Program to interface

03 List

ArrayList · sort · filter · stats · LinkedList

04 Set

HashSet · LinkedHashSet · TreeSet · Demo

05 Map

HashMap · TreeMap · CRUD · entrySet

06 Queue & Iterator

Queue · Deque · PriorityQueue · Iterator

Why Do We Need Collections?

In the Arrays chapter you built `count + ensureCapacity` by hand. Collections give you ready-made dynamic structures and rich APIs.

Dynamic size

Grow and shrink without rewriting `ensureCapacity()`

Standard APIs

`add` · `remove` · `contains` · `iterate` — same ideas everywhere

Right structure

List, Set, or Map — pick the contract you need

Algorithms

`Collections.sort`, `binarySearch`, ... on List

1 · INTRODUCTION TO COLLECTIONS

Arrays vs Collections · The Framework

From hand-built dynamic arrays to standard library types

Arrays vs Collections

Array

- Fixed length after creation
- length field (capacity)
- You track count yourself
- Fast index access $O(1)$
- Primitive or object types
- Insert/delete in middle = shift
- No rich standard API

Collection

- Dynamic size (grow / shrink)
- size() = logical count
- Framework manages storage
- Access depends on type
- Objects only (use wrappers)
- Methods: add, remove, contains...
- Shared interfaces & algorithms

Same Job — Two Styles

```
// Array + count (your IntArray)
```

```
Employee[] staff = new Employee[10];
```

```
int count = 0;
```

```
staff[count++] = e;
```

```
// grow by copy when full ...
```

```
for (int i = 0; i < count; i++)
```

```
    System.out.println(staff[i]);
```

```
// Collection — ArrayList
```

```
List<Employee> staff =
```

```
    new ArrayList<>();
```

```
staff.add(e);
```

```
// grows automatically
```

```
for (Employee e : staff)
```

```
    System.out.println(e);
```

Quick Comparison

Aspect	Array	Collection
Size	Fixed length	Dynamic
Primitives	Yes (int[])	No — use Integer, ...
API	[] and length	add, remove, size, ...
Type safety	Element type	Generics <E>
When to use	Fixed, primitives, speed	Growing data, rich ops

What Is the Collections Framework?

A set of interfaces and classes in `java.util` that represent and manipulate groups of objects — with a unified design.

Interfaces

`Collection`, `List`, `Set`, `Queue`, `Map` — the contracts

Implementations

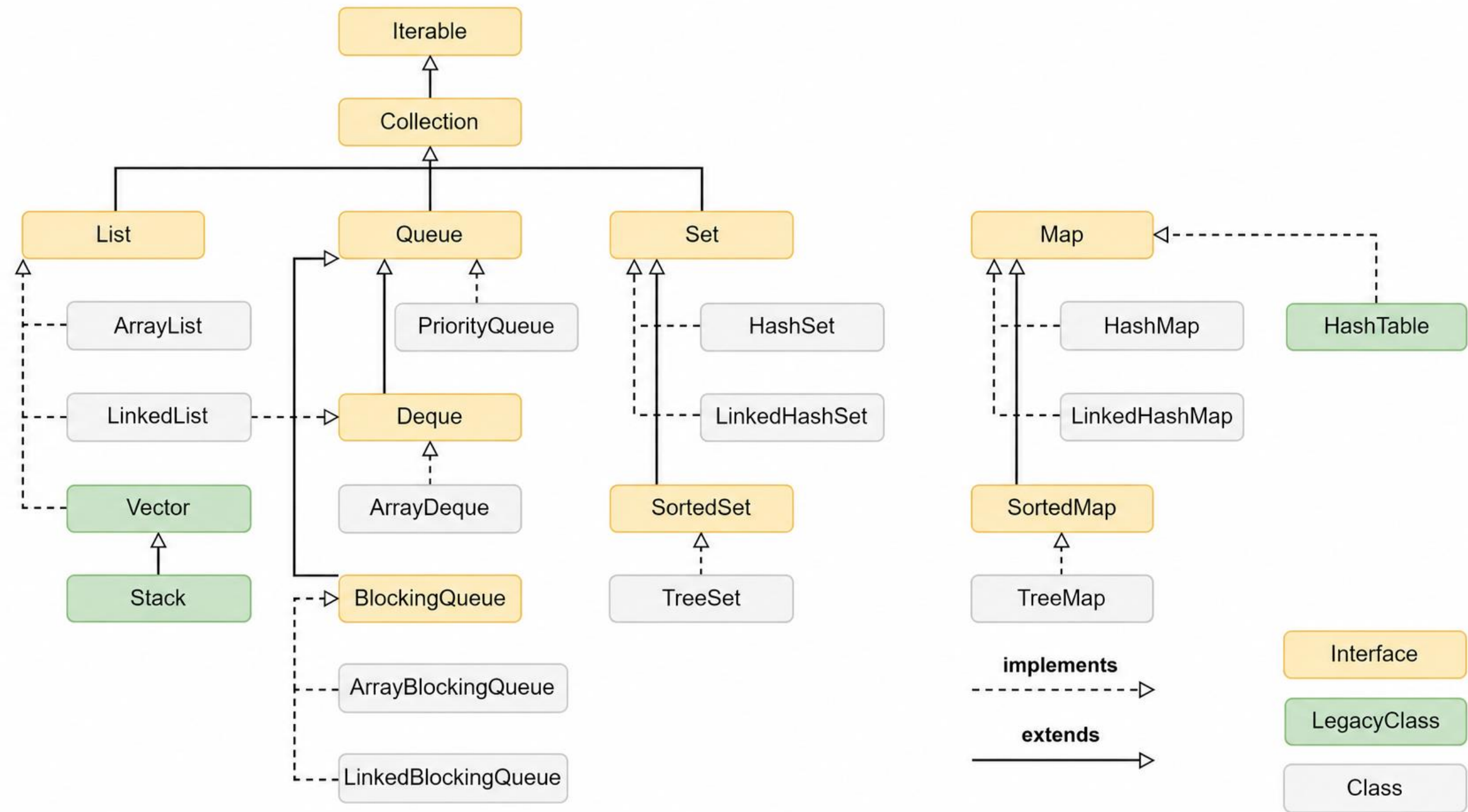
`ArrayList`, `HashSet`, `HashMap`, ... — the concrete types

Algorithms

`Collections.sort`, `binarySearch`, `shuffle`, ...

Package

`java.util` (and `java.util.concurrent` for concurrent variants)



Benefits of the Framework

Reusable design

Learn List once — ArrayList and LinkedList share the API

Interoperability

Pass List where Collection is expected

Performance choices

Pick ArrayList or LinkedList for access vs insert patterns

Less boilerplate

No hand-written ensureCapacity for everyday work

Algorithms

Sort, search, min/max on any suitable List

Collections Architecture

The framework is layered: interfaces define contracts, concrete classes implement them, and utility classes provide algorithms.

1 · Interfaces

Collection, List, Set, Queue, Map, Iterator...

2 · Implementations

ArrayList, LinkedList, HashSet, HashMap, TreeMap...

3 · Algorithms

Collections.sort, binarySearch, shuffle, min, max...

4 · Infrastructure

Iterator, Comparable, Comparator, equals / hashCode

How the Pieces Fit

```
Your code · List<Employee> staff = new ArrayList<>();
```

Interface

List<E>
(contract)

Implementation

ArrayList<E>
(array-backed)

Algorithms

Collections.sort
binarySearch

Program to the interface (List), choose an implementation (ArrayList), then use shared algorithms when needed.

Core Collection Interfaces

Five main interfaces you will use every day. Map is not a subtype of Collection, but belongs to the same framework.

Collection<E>

Root of the hierarchy — group of elements

List<E>

Ordered sequence · allows duplicates · index access

Set<E>

No duplicates · at most one null (usually)

Queue<E>

Hold elements for processing · FIFO typical

Map<K,V>

Key-value pairs · unique keys

Collection<E> — Common Operations

```
// Most lists and sets share these ideas
```

```
boolean add(E e);
```

```
boolean remove(Object o);
```

```
boolean contains(Object o);
```

```
int size();
```

```
boolean isEmpty();
```

```
void clear();
```

```
Iterator<E> iterator();
```

```
// Plus bulk ops: addAll, removeAll, retainAll, containsAll
```

List · Set · Map — When to Use

List

- Ordered
- Duplicates OK
- Index access
- Employee roster
- History of actions

ArrayList

LinkedList

Set

- Unique elements
- No duplicates
- Membership tests
- Tags, IDs seen
- Unique employeeIds

HashSet

TreeSet

Map

- Key → value
- Unique keys
- Fast lookup by key
- id → Employee
- Config settings

HashMap

TreeMap

Queue — Processing Order

A Queue holds elements before processing. Typical order is FIFO (first in, first out). Deque allows both ends.

offer / add

Insert at the tail (depending on capacity rules)

poll / remove

Take from the head

peek / element

Look at the head without removing

Use cases

Task queues, buffering, BFS, message handling

Program to the Interface

```
// Prefer interface type on the left
```

```
List<Employee> staff = new ArrayList<>();
```

```
Set<String> ids = new HashSet<>();
```

```
Map<String, Employee> byId = new HashMap<>();
```

```
// Why?
```

```
// 1. Callers depend on the contract, not the class
```

```
// 2. Easy to switch ArrayList ↔ LinkedList later
```

```
// 3. Methods can accept List / Collection / Map
```

Three Easy-to-Confuse Names

Collection

Interface — root of List, Set, Queue (`java.util.Collection`)

Collections

Utility class — static methods: `sort`, `binarySearch`, ...

Map

Interface — key-value store; NOT a subtype of Collection

Collection vs Collections

Collection (interface)

- Type of an object
- List / Set implement it
- Instance methods:
add, remove, size...

```
List<Employee> list =  
    new ArrayList<>();  
list.add(e);
```

Collections (class)

- Utility class only
- All methods static
- Works on Collection / List

```
Collections.sort(list);  
Collections.binarySearch(  
    list, key);  
Collections.shuffle(list);
```

Where Map Fits

Map is part of the framework but does not extend Collection. You cannot write `Collection c = new HashMap<>()`.

```
List<Employee> staff = new ArrayList<>(); // Collection
```

```
Set<String> ids = new HashSet<>();      // Collection
```

```
Map<String, Employee> byId = new HashMap<>(); // Map
```

```
Collections.sort(staff); // utility class
```

DEMO · EMPLOYEE COLLECTION

Same domain as the Arrays chapter

Unique employeeId on every add

Employee — Element Type

```
public class Employee {  
    private String employeeId;  
    private String name;  
  
    // Constructors/ setters/ getters/ toString  
}
```

findEmployeeById — Linear Search

```
List<Employee> staff = new ArrayList<>();
```

```
Employee findEmployeeById(String employeeId) {  
    for (Employee e : staff) {  
        if (e.getEmployeeId().equals(employeeId))  
            return e;  
    }  
    return null; // not found  
}
```

```
// O(n) — simple, clear, one source of truth (the List)
```

addEmployee — No Duplicate Id

```
void addEmployee(Employee e) {  
    if (e == null)  
        throw new IllegalArgumentException("null");  
    if (findEmployeeById(e.getEmployeeId()) != null)  
        throw new IllegalStateException("duplicate id");  
    staff.add(e);  
}  
  
addEmployee(new Employee("E01", "An"));  
addEmployee(new Employee("E02", "Binh"));  
// addEmployee(new Employee("E01", "Other")); // throws
```

Two Ways to Reject Duplicate Ids

A · Scan the List

```
if (findEmployeeById(  
    e.getEmployeeId()) != null)  
    throw ...("duplicate id");  
staff.add(e);
```

// findById scans List → $O(n)$

// One source of truth

B · HashSet of ids

```
Set<String> usedIds =  
    new HashSet<>();  
if (!usedIds.add(  
    e.getEmployeeId()))  
    throw ...("duplicate id");  
staff.add(e);  
  
// average  $O(1)$   
// Keep Set in sync on remove
```

Which Is More Optimal?

A · findById**B · HashSet****Time per add** $O(n)$ list scan $O(1)$ average**n adds total** $\approx O(n^2)$ $\approx O(n)$ **Extra memory**

None

One Set of ids

Source of truth

Only the List

List + Set (sync)

Best when

Small n · teaching

Large n · many inserts

Recommendation

Both are correct

A is clear and fine for small lists and learning

B scales better

Avoid $O(n^2)$ when many employees are added

Often best: Map

`Map<String, Employee> byId` — $O(1)$ lookup and unique keys

If you use Set + List

Update both on add and remove — stay consistent

2 · GENERICS WITH COLLECTIONS

Type-safe lists, sets, and maps

Why generics · `List<E>` · `Set<E>` · `Map<K,V>`

Why Generics?

Before generics, collections stored `Object`. You cast on every get — and mistakes showed up only at runtime.

Compile-time safety

Wrong type → compile error, not `ClassCastException` later

No casts

`get()` returns `Employee`, not `Object`

Clearer intent

`List<Employee>` documents what the list holds

Same idea as `Manager<E>`

You already saw generics with the dynamic array pattern

Before vs After Generics

Before (raw / Object)

```
List list = new ArrayList();  
list.add(new Employee(...));  
list.add("oops"); // allowed!
```

```
Employee e =  
    (Employee) list.get(0);  
// ClassCastException risk
```

After (generic)

```
List<Employee> list =  
    new ArrayList<>();  
list.add(new Employee(...));  
// list.add("oops");  
// compile error
```

```
Employee e = list.get(0);  
// no cast
```

Generic Collection Types

List<E>

List of elements of type E — List<Employee>

Set<E>

Set of unique E — Set<String> for employeeIds

Queue<E>

Queue of E — Queue<Task>

Map<K, V>

Keys of type K, values of type V — Map<String, Employee>

Diamond <>

new ArrayList<>() — compiler infers the type argument

Declaring Generic Collections

```
List<Employee> staff = new ArrayList<>();
```

```
Set<String> ids = new HashSet<>();
```

```
Map<String, Employee> byId = new HashMap<>();
```

```
// Interface on the left · concrete class on the right
```

```
List<String> names = new LinkedList<>();
```

```
// Type parameter must be a reference type
```

```
// List<int> → illegal
```

```
List<Integer> scores = new ArrayList<>();
```

No Primitives — Use Wrappers

Type parameters must be reference types. Java boxes primitives automatically when you use wrapper types.

```
List<Integer> nums = new ArrayList<>();
```

```
nums.add(10);    // autobox int → Integer
```

```
int x = nums.get(0); // unbox Integer → int
```

```
// Common wrappers: Integer, Double, Boolean, Character, Long
```

Employee Domain — Typed Collections

```
List<Employee> staff = new ArrayList<>();
```

```
staff.add(new Employee("E01", "An"));
```

```
Employee first = staff.get(0); // typed
```

```
Map<String, Employee> byId = new HashMap<>();
```

```
byId.put("E01", first);
```

```
Employee e = byId.get("E01");
```

```
Set<Employee> unique = new HashSet<>(staff);
```

```
// needs equals/hashCode on Employee for correctness
```

Type Safety with Generics

The compiler checks that only the declared element type enters the collection — mistakes become compile errors.

Add only E

List<Employee> rejects String at compile time

Get returns E

No cast — Employee e = list.get(0)

Raw types

List without <E> disables safety — avoid them

Heap pollution

Mixing raw and generic can still break at runtime

What the Compiler Blocks

```
List<Employee> staff = new ArrayList<>();  
staff.add(new Employee("E01", "An")); // OK
```

```
staff.add("hello");  
// error: incompatible types: String cannot be  
//      converted to Employee
```

```
Employee e = staff.get(0); // OK — type Employee  
// String s = staff.get(0); // also a compile error
```

Avoid Raw Types

```
// Raw type — compiler warning, weak safety
```

```
List raw = new ArrayList();
```

```
raw.add(new Employee("E01", "An"));
```

```
raw.add("not an employee"); // allowed!
```

```
List<Employee> staff = raw; // unchecked warning
```

```
Employee e = staff.get(1); // ClassCastException!
```

```
// Rule: always write List<Employee>, never plain List
```

The Diamond Operator <>

Since Java 7, <> tells the compiler to infer the type arguments from the left-hand side (or the context).

```
// Verbose (still valid)
```

```
List<Employee> staff =  
    new ArrayList<Employee>();
```

```
// Diamond — preferred
```

```
List<Employee> staff = new ArrayList<>();  
Map<String, Employee> byId = new HashMap<>();
```

Where Diamond Works

```
// Variable declaration
```

```
Set<String> ids = new HashSet<>();
```

```
// Return / assign from method context (Java 9+ improved)
```

```
List<Employee> empty() {  
    return new ArrayList<>();  
}
```

```
// Still write the type on the left (or method return)
```

```
// <> alone is not a type — needs context to infer
```

Why Use <> ?

Less repetition

Type appears once on the left, not twice

Easier refactor

Change `List<Employee>` → `List<Staff>` in one place

Same bytecode idea

Inference is compile-time only — runtime is erased

Style

Modern Java code almost always uses the diamond

Setup — Diamond + Typed Lists

```
List<Employee> staff = new ArrayList<>();
```

```
Map<String, Employee> byId = new HashMap<>();
```

```
Set<String> usedIds = new HashSet<>();
```

```
// All use diamond <> — types inferred from the left
```

```
// staff holds only Employee · byId keys are String
```

Type-Safe add

```
Employee an = new Employee("E01", "An");  
staff.add(an);           // OK  
byId.put(an.getEmployeeId(), an);  
usedIds.add(an.getEmployeeId());  
  
// staff.add("hello");  
// COMPILE ERROR — String is not Employee  
  
Employee e = staff.get(0); // no cast  
String id = e.getEmployeeId();
```

findEmployeeById — Returns Employee

```
Employee findEmployeeById(String employeeId) {  
    return byId.get(employeeId); // Employee or null  
}
```

// Or scan the list ($O(n)$) — still typed:

```
Employee findInList(String employeeId) {  
    for (Employee e : staff) {  
        if (e.getEmployeeId().equals(employeeId)) return e;  
    }  
    return null;  
}
```

3 · LIST

Ordered sequences with index access

Interface · characteristics · ArrayList · LinkedList

What Is List?

List is an ordered Collection that allows duplicates and access by integer index — the closest standard type to a dynamic array of objects.

Extends Collection

Inherits add, remove, size, iterator, ...

Positional access

get(i), set(i), add(i, e), remove(i)

Search

indexOf, lastIndexOf

ListIterator

Bidirectional traversal, add/set while iterating

Core List Operations

```
List<Employee> staff = new ArrayList<>();
```

```
staff.add(e);           // append  
staff.add(0, e);        // insert at index  
Employee x = staff.get(i); // read  
staff.set(i, e);        // replace  
staff.remove(i);        // by index  
staff.remove(e);        // by equals  
int i = staff.indexOf(e); // -1 if absent  
int n = staff.size();
```

List Characteristics

Ordered

Elements keep insertion order (unless you sort later)

Indexed

Positions 0 .. size()-1 — like an array

Duplicates allowed

Two Employees with different ids can both be in the list

Null

ArrayList allows null elements (use carefully)

Dynamic size

Grows and shrinks — no fixed length field to manage

List vs Array (Reminder)

Employee[]**List<Employee>****Size**

Fixed length

Dynamic size()

Access

a[i]

list.get(i)

Append

manual + grow

list.add(e)

Insert mid

shift yourself

list.add(i, e)

Type

array type

interface + impl

Order and Index — Mental Model

staff → [E01-An | E02-Binh | E03-Chi]

index 0 1 2

get(1)

→ E02-Binh

add(1, x)

inserts, shifts right

remove(0)

shifts left

size()

→ 3

When to Use a List

Good fit

- Need order (roster, history, playlist)
- Need index access or insert at position
- Duplicates are meaningful
- Replacing your hand-built `Employee[]` + count

Consider something else

- Only care about uniqueness → prefer Set
- Primary access by key → prefer Map
- Fixed primitive buffer → array may still win

ArrayList — Resizable Array

ArrayList is the most common List implementation: a dynamic array of object references. Same idea as your `Employee[]` + `count` + `ensureCapacity`.

Backed by Object[]

Contiguous slots · fast random access

Grows automatically

Typically ~1.5× capacity when full

Implements List

All List operations + RandomAccess

Not synchronized

Single-threaded by default (use concurrent types later)

Domain — Polymorphic Employees

«abstract» Employee (#employeeId, #name, +pay() abstract)

△

└── FullTimeEmployee (-monthlySalary, +pay())

└── PartTimeEmployee (-hourlyRate, -hoursWorked, +pay())

List<Employee> holds both subtypes — pay() uses dynamic dispatch.

Create and Add (Polymorphic)

```
List<Employee> staff = new ArrayList<>();
```

```
staff.add(new FullTimeEmployee("E01", "An", 1500));
```

```
staff.add(new PartTimeEmployee("E02", "Binh", 10, 80));
```

```
staff.add(new FullTimeEmployee("E03", "Chi", 1800));
```

```
staff.size();    // 3
```

```
staff.get(1);    // PartTime E02
```

```
// Index access O(1) — array-backed
```

ArrayList — Typical Complexity

Operation	Cost	Note
<code>get(i) / set(i)</code>	$O(1)$	Direct index into array
<code>add(e)</code> at end	$O(1)^*$	*amortized; grow sometimes
<code>add(i, e) / remove(i)</code>	$O(n)$	Shift elements
<code>contains / indexOf</code>	$O(n)$	Linear scan
<code>size()</code>	$O(1)$	Stored count

Polymorphism on the List

```
double totalPayroll = 0;
for (Employee e : staff) {
    totalPayroll += e.pay(); // dynamic dispatch
}

// FullTime → monthlySalary
// PartTime → hourlyRate * hoursWorked

System.out.println(totalPayroll);
```

Capacity vs size()

`size()` is the number of elements. Capacity is the length of the internal array — grows when needed (like `ensureCapacity`).

```
List<Employee> staff = new ArrayList<>(20); // initial capacity  
staff.add(...); // size increases; capacity may stay 20
```

```
// Optional: trimToSize() on ArrayList to shrink capacity  
// Optional: ensureCapacity(n) before a bulk of adds
```

When to Prefer ArrayList

Default choice for List

Most teaching and business code starts here

Frequent `get(i)` / `set(i)`

$O(1)$ random access

Append-heavy workloads

add at end is cheap amortized

Iteration

Excellent cache locality (contiguous array)

Less ideal

Many inserts/removes in the middle → consider LinkedList

DEMO · ARRAYLIST OPERATIONS

Add · Remove one · Remove many

List<Employee> with FullTime & PartTime

add(index, element) — Insert

```
// Insert at position — shifts the rest right O(n)
Employee chi = new FullTimeEmployee("E03", "Chi", 1800);
if (findById(chi.getEmployeeId()) != null) throw ...;
staff.add(1, chi); // index 1
```

```
// Before: [An, Binh]
```

```
// After: [An, Chi, Binh]
```

```
staff.add(0, e); // front
```

```
staff.add(staff.size(), e); // same as add(e)
```

Remove One — by Index

```
Employee removed = staff.remove(1); // by index  
// shifts left · returns the removed element
```

```
boolean ok = staff.remove(anEmployee); // by equals  
// needs equals/hashCode on Employee for value equality
```

```
// Prefer remove by id for our domain:
```

```
boolean removeById(String employeeId) {  
    int i = indexOfId(employeeId);  
    if (i < 0) return false;  
    staff.remove(i);  
    return true;  
}
```

Remove One — by Name (First Match)

```
boolean removeFirstByName(String name) {  
    for (int i = 0; i < staff.size(); i++) {  
        if (staff.get(i).getName().equals(name)) {  
            staff.remove(i); // remove one  
            return true;  
        }  
    }  
    return false; // none found  
}  
  
removeFirstByName("An"); // removes first An only
```

Remove Many — by Name (All Matches)

// Wrong: for-each + remove → `ConcurrentModificationException`

// Wrong: forward index without adjusting after remove

```
int removeAllByName(String name) {  
    int removed = 0;  
    for (int i = staff.size() - 1; i >= 0; i--) {  
        if (staff.get(i).getName().equals(name)) {  
            staff.remove(i);  
            removed++;  
        }  
    }  
    return removed;  
}
```

Remove Many — removeIf (Cleaner)

```
// Java 8+ — safe bulk remove by predicate
```

```
staff.removeIf(e -> e.getName().equals("An"));
```

```
// All part-time employees
```

```
staff.removeIf(e -> e instanceof PartTimeEmployee);
```

```
// Low pay
```

```
staff.removeIf(e -> e.pay() < 500);
```

```
// Returns boolean: true if any element was removed
```

Add & Remove — Summary

Goal

Approach

Append

`add(e)` after unique-id check

Insert at *i*

`add(i, e)` — $O(n)$ shift

Remove one by id

`indexOfId + remove(i)`

Remove one by name

`scan · remove(i) · return`

Remove all by criteria

backward loop or `removeIf`

Two Ways to Update

1 · Replace reference

```
list.set(i, newObject)
```

Slot points to a different instance.
Old object may become unreachable.

Use when type or whole state changes.

2 · Mutate fields

```
e.setName(...) / setters
```

Same object in the list.
Only some attributes change.

Use when employeeId stays the same.

Update — Replace the Reference

```
boolean replaceById(String employeeId, Employee neu) {  
    if (neu == null) throw new IllegalArgumentException();  
    // Keep id stable — domain rule  
    if (!employeeId.equals(neu.getId()))  
        throw new IllegalArgumentException("id mismatch");  
    int i = indexOfId(employeeId);  
    if (i < 0) return false;  
    staff.set(i, neu); // new reference in the slot  
    return true;  
}  
  
// PartTime → FullTime at same id  
replaceById("E02", new FullTimeEmployee("E02", "Binh", 1600));
```

Update — Only Some Attributes

```
// Same object — change fields via setters on the hierarchy
```

```
boolean updateName(String employeeId, String newName) {
```

```
    Employee e = findById(employeeId);
```

```
    if (e == null) return false;
```

```
    e.setName(newName); // mutate
```

```
    return true;
```

```
}
```

```
boolean updateFullTimeSalary(String id, double salary) {
```

```
    Employee e = findById(id);
```

```
    if (!(e instanceof FullTimeEmployee ft)) return false;
```

```
    ft.setMonthlySalary(salary);
```

```
    return true;
```

```
}
```

Replace vs Mutate

`set(i, new Emp)`

`mutate fields`

List slot

New reference

Same reference

employeeId

Must match (our rule)

Unchanged

Type change

Yes (PT → FT)

No — same class

Other refs

Still see old object

See the change

API

`staff.set(i, neu)`

`e.setName(...)`

Mental Model

Replace: `staff[i]` —X—► old Employee
 `staff[i]` —————► new Employee

Mutate: `staff[i]` —————► same Employee (fields changed)

If another variable still references the old object, replace does not change what that variable sees. Mutate does — everyone sharing the instance sees the new field values.

Find One · Filter Many

Find one

Return Employee or null — first match by id / name / type

Filter many

Return List<Employee> — all matches (possibly empty)

Combined criteria

AND / OR on name, type, pay(), ...

Does not modify staff

Unlike remove — original list stays unchanged

Find One — First Match by Name

```
Employee findFirstByName(String name) {  
    for (Employee e : staff) {  
        if (e.getName().equals(name))  
            return e;  
    }  
    return null;  
}
```

// Names need not be unique — only the first hit is returned

Filter Many — by Name

```
List<Employee> filterByName(String name) {  
    List<Employee> result = new ArrayList<>();  
    for (Employee e : staff) {  
        if (e.getName().equals(name))  
            result.add(e);  
    }  
    return result; // empty list if none  
}  
  
List<Employee> anList = filterByName("An");  
for (Employee e : anList) System.out.println(e);
```

Filter Many — by Type

```
List<Employee> filterPartTime() {  
    List<Employee> result = new ArrayList<>();  
    for (Employee e : staff) {  
        if (e instanceof PartTimeEmployee)  
            result.add(e);  
    }  
    return result;  
}  
  
// Or stream (preview):  
staff.stream()  
    .filter(e -> e instanceof PartTimeEmployee)  
    .toList();
```

Combined Criteria — AND

```
// Full-time AND pay >= minPay
List<Employee> filterFullTimeMinPay(double minPay) {
    List<Employee> result = new ArrayList<>();
    for (Employee e : staff) {
        if (e instanceof FullTimeEmployee
            && e.pay() >= minPay) {
            result.add(e);
        }
    }
    return result;
}
```

Combined Criteria — Flexible Predicate

// General filter — caller passes the condition

```
List<Employee> filter(Predicate<Employee> test) {
```

```
    List<Employee> result = new ArrayList<>();
```

```
    for (Employee e : staff) {
```

```
        if (test.test(e)) result.add(e);
```

```
    }
```

```
    return result;
```

```
}
```

// Examples

```
filter(e -> e.getName().startsWith("A"));
```

```
filter(e -> e.pay() < 500 || e instanceof PartTimeEmployee);
```

Find vs Filter

Find one

Filter many

Return type

Employee (or null)

List<Employee>

Matches

First only

All

Empty case

null

Empty list

Modifies staff?

No

No

Typical use

by id

by name / type / pay

Why Sort Manually First?

See the idea

Compare elements, swap, grow a sorted prefix

Same as Arrays chapter

Selection sort on `int[]` → now on `List<Employee>`

Then use the library

`Collections.sort` / `stream().sorted()` hide the loops

Complexity

Feel $O(n^2)$ before trusting $O(n \log n)$ library sorts

Selection Sort — Idea

For each index i , find the minimum in the range $i..n-1$, then swap it with position i . The left part grows into a sorted prefix.

$i = 0$

Find min in whole list → swap to index 0

$i = 1$

Find min in $1..n-1$ → swap to index 1

...

Repeat until $i = n-2$

Cost

$\approx n^2/2$ comparisons — $O(n^2)$

Selection Sort — by Name

```
void selectionSortByName() {  
    for (int i = 0; i < staff.size() - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < staff.size(); j++) {  
            if (staff.get(j).getName()  
                .compareTo(staff.get(min).getName()) < 0)  
                min = j;  
        }  
        // swap i and min  
        Employee tmp = staff.get(i);  
        staff.set(i, staff.get(min));  
        staff.set(min, tmp);  
    }  
}
```

Selection Sort — by pay()

```
void selectionSortByPay() {  
    for (int i = 0; i < staff.size() - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < staff.size(); j++) {  
            if (staff.get(j).pay() < staff.get(min).pay())  
                min = j;  
        }  
        Employee tmp = staff.get(i);  
        staff.set(i, staff.get(min));  
        staff.set(min, tmp);  
    }  
}  
  
// Polymorphic pay() — FullTime & PartTime both work
```

Swap Helper on List

```
void swap(int i, int j) {
```

```
    Employee tmp = staff.get(i);
```

```
    staff.set(i, staff.get(j));
```

```
    staff.set(j, tmp);
```

```
}
```

```
// Then in selection sort:
```

```
swap(i, min);
```

```
// List has no a[i] — use get / set
```

Bubble Sort — Brief Look

```
void bubbleSortByName() {  
    for (int pass = 0; pass < staff.size() - 1; pass++) {  
        for (int j = 0; j < staff.size() - 1 - pass; j++) {  
            if (staff.get(j).getName()  
                .compareTo(staff.get(j + 1).getName()) > 0) {  
                swap(j, j + 1); // adjacent  
            }  
        }  
    }  
}
```

// Also $O(n^2)$ — good for learning, not for large n

From Manual to Library

You wrote

Nested loops · `compareTo` / `pay()` · swap via `get/set`

Library does

`Collections.sort` / `list.sort` / `stream().sorted` — $O(n \log n)$

Same Comparator idea

You decide the order; the algorithm is provided

Next slides

Sort in place · sorted copy · Stream pipelines

Manual Multi-Criteria Sort

Same selection-sort loops. Only the comparison changes: try key1; if equal, try key2 (and maybe key3).

`compare(a, b) < 0`

a should come before b

Key order

Primary first; secondary only when primary ties

Reuse

One compare method · call it from selection sort

compare — Name then employeeId

```
/** @return negative if a before b, 0 if equal, positive if a after b */  
int compareNameThenId(Employee a, Employee b) {  
    int c = a.getName().compareTo(b.getName());  
    if (c != 0) return c; // primary decides  
    return a.getEmployeeId().compareTo(b.getEmployeeId());  
}
```

// Same logic as thenComparing — written by hand

Selection Sort + Multi-Key compare

```
void selectionSortNameThenId() {  
    for (int i = 0; i < staff.size() - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < staff.size(); j++) {  
            if (compareNameThenId(staff.get(j), staff.get(min)) < 0)  
                min = j;  
        }  
        swap(i, min);  
    }  
}
```

compare — pay asc, then Name

```
int comparePayThenName(Employee a, Employee b) {  
    int c = Double.compare(a.pay(), b.pay()); // primary  
    if (c != 0) return c;  
    return a.getName().compareTo(b.getName()); // tie-break  
}
```

```
// Descending pay: flip primary  
int c = Double.compare(b.pay(), a.pay()); // higher pay first
```

Two Ways to Sort

A · Sort in place

```
Collections.sort(staff, cmp)
```

- Changes the list order
- No extra list
- void-style API
- Callers see new order

Use when staff is yours
and order should stick.

B · Return sorted copy

new list + sort

- Original order kept
- Extra memory $O(n)$
- Returns `List<Employee>`
- Safe for shared lists

Use when you only need
a view sorted for display.

A — Sort in Place

```
import java.util.Collections;
import java.util.Comparator;

void sortByName() {
    Collections.sort(staff,
        Comparator.comparing(Employee::getName));
}

void sortByPayDescending() {
    staff.sort(Comparator.comparingDouble(Employee::pay)
        .reversed());
}

// staff order is permanently changed
```

B — Return a Sorted Copy

```
List<Employee> sortedByName() {
```

```
    List<Employee> copy = new ArrayList<>(staff);
```

```
    copy.sort(Comparator.comparing(Employee::getName));
```

```
    return copy; // staff unchanged
```

```
}
```

```
List<Employee> sortedByPayDesc() {
```

```
    List<Employee> copy = new ArrayList<>(staff);
```

```
    copy.sort(Comparator.comparingDouble(Employee::pay).reversed());
```

```
    return copy;
```

```
}
```

In Place vs Sorted Copy

Sort in place

Return sorted copy

Original order

Lost

Preserved

Memory $O(1)$ extra $O(n)$ new list**Return type**

void

List<Employee>

Shared list

Risky

Safe

Typical use

Internal roster order

Report / UI snapshot

Which Should You Choose?

Own the list · order matters going forward

Sort in place — `Collections.sort` / `list.sort`

Only need a sorted view once

Copy then sort — original stays stable

Multiple sort keys over time

Copy for each view, or sort in place before use

API design

Returning a list signals “here is a result”; void signals “I changed state”

Three Keys — Chain thenComparing

```
// Type group (FullTime first), then pay desc, then name
staff.sort(
  Comparator.comparing((Employee e) ->
    e instanceof FullTimeEmployee ? 0 : 1)
    .thenComparing(Comparator.comparingDouble(Employee::pay).reversed())
    .thenComparing(Employee::getName)
  );
```

Sorting with Stream API

`stream().sorted(...)` builds a new ordered sequence. The original list is not modified — same idea as “return a sorted copy”.

Non-mutating

staff stays in its current order

Chainable

`filter` → `sorted` → `map` → `toList`

Comparator

Same `Comparator.comparing` as `List.sort`

sorted by Name

```
List<Employee> byName = staff.stream()  
    .sorted(Comparator.comparing(Employee::getName))  
    .toList(); // Java 16+ unmodifiable
```

// Or mutable list:

```
List<Employee> byName = staff.stream()  
    .sorted(Comparator.comparing(Employee::getName))  
    .collect(Collectors.toList());
```

// staff itself is unchanged

sorted by pay() and employeeId

```
// Highest pay first
```

```
List<Employee> byPay = staff.stream()
```

```
    .sorted(Comparator.comparingDouble(Employee::pay).reversed())
```

```
    .toList();
```

```
List<Employee> byId = staff.stream()
```

```
    .sorted(Comparator.comparing(Employee::getEmployeeId))
```

```
    .toList();
```

Filter then sorted — Pipeline

```
// Part-time only, sorted by name
```

```
List<Employee> partTimeSorted = staff.stream()  
    .filter(e -> e instanceof PartTimeEmployee)  
    .sorted(Comparator.comparing(Employee::getName))  
    .toList();
```

```
// Full-time with pay >= 1500, highest pay first
```

```
staff.stream()  
    .filter(e -> e instanceof FullTimeEmployee && e.pay() >= 1500)  
    .sorted(Comparator.comparingDouble(Employee::pay).reversed())  
    .toList();
```

Stream.sorted vs List.sort

	List.sort / Collections.sort	stream().sorted()
Mutates stuff?	Yes	No
Result	void (same list)	New stream → collect list
Chain filter	Separate step	Natural in pipeline
Best when	Permanent new order	One-off sorted view

Statistics on List<Employee>

Count, sum, average — on all elements or only a subtype. Polymorphism for pay(); downcast when you need subtype fields.

No condition

Whole list — size, total payroll, average pay

With condition

Only FullTime / PartTime / pay $\geq$ threshold

Upcast

FullTime stored as Employee in the list

Downcast

instanceof then (FullTimeEmployee) to read monthlySalary

No Condition — All Employees

```
int countAll() {  
    return staff.size();  
}  
  
double totalPayroll() { // polymorphic pay()  
    double sum = 0;  
    for (Employee e : staff) sum += e.pay(); // upcast ref  
    return sum;  
}  
  
double averagePay() {  
    if (staff.isEmpty()) return 0;  
    return totalPayroll() / staff.size();  
}
```

Upcasting — Already in the List

```
List<Employee> staff = new ArrayList<>();  
staff.add(new FullTimeEmployee("E01", "An", 1500)); // upcast  
staff.add(new PartTimeEmployee("E02", "Binh", 10, 80));  
  
Employee e = staff.get(0); // static type Employee  
e.pay();                  // dynamic dispatch  
  
// e.getMonthlySalary(); // COMPILE ERROR — not on Employee  
// Need downcast to reach subtype members
```

With Condition — Count by Type

```
int countFullTime() {  
    int n = 0;  
    for (Employee e : staff) {  
        if (e instanceof FullTimeEmployee) n++;  
    }  
    return n;  
}  
  
int countPartTime() {  
    int n = 0;  
    for (Employee e : staff)  
        if (e instanceof PartTimeEmployee) n++;  
    return n;  
}
```

Downcast — Subtype Fields

```
// Sum of monthlySalary — only on FullTimeEmployee
double totalMonthlySalary() {
    double sum = 0;
    for (Employee e : staff) {
        if (e instanceof FullTimeEmployee ft) { // pattern match
            sum += ft.getMonthlySalary(); // downcast
        }
    }
    return sum;
}
```

```
// Classic style: (FullTimeEmployee) e after instanceof
```

Downcast — PartTime hours

```
int totalHoursWorked() {  
    int hours = 0;  
    for (Employee e : staff) {  
        if (e instanceof PartTimeEmployee pt) {  
            hours += pt.getHoursWorked();  
        }  
    }  
    return hours;  
}
```

// Without downcast you only have Employee methods

Conditional Aggregate — pay threshold

```
double averageFullTimePay() {  
    double sum = 0; int n = 0;  
    for (Employee e : staff) {  
        if (e instanceof FullTimeEmployee) {  
            sum += e.pay(); n++;  
        }  
    }  
    return n == 0 ? 0 : sum / n;  
}
```

Same Stats with Stream (Preview)

```
long fullTimeCount = staff.stream()  
    .filter(e -> e instanceof FullTimeEmployee)  
    .count();
```

```
double totalPay = staff.stream()  
    .mapToDouble(Employee::pay)  
    .sum();
```

```
double sumSalary = staff.stream()  
    .filter(FullTimeEmployee.class::isInstance)  
    .map(FullTimeEmployee.class::cast)  
    .mapToDouble(FullTimeEmployee::getMonthlySalary)  
    .sum();
```

When Do You Need a Cast?

Need

Cast?

`e.pay() / getName() / getEmployeeId()`

No — on Employee

`Count FullTime / PartTime`

`instanceof` only

`getMonthlySalary()`

Yes — downcast to FullTime

`getHoursWorked()`

Yes — downcast to PartTime

Total payroll of everyone

No — polymorphic `pay()`

4 · COLLECTION TRAVERSAL

How to walk every element

Traditional for · Enhanced for · forEach()

Same List — Three Traversals

```
// 1) Traditional — with index
for (int i = 0; i < staff.size(); i++)
    System.out.println(i + " " + staff.get(i).getName());

// 2) Enhanced — polymorphic pay
for (Employee e : staff)
    System.out.println(e.getName() + " → " + e.pay());

// 3) forEach
staff.forEach(e -> System.out.println(e));
```

Lambda Expression with Collections

A lambda is a short form of a single-method object. Collection APIs accept functional interfaces: Consumer, Predicate, Comparator.

Consumer<T>

forEach — action on each element

Predicate<T>

removeIf — true means remove

Comparator<T>

sort — order of two elements

Lambda Syntax (Quick)

```
// (parameters) -> body
```

```
e -> System.out.println(e)
```

```
(a, b) -> a.getName().compareTo(b.getName())
```

```
// Block body
```

```
e -> {
```

```
    System.out.println(e.getName());
```

```
    System.out.println(e.pay());
```

```
}
```

```
// Method reference
```

```
System.out::println
```

```
Employee::getName
```

forEach with Lambda

```
staff.forEach(e -> System.out.println(e));
```

```
staff.forEach(e ->  
    System.out.println(e.getName() + " → " + e.pay()));
```

```
staff.forEach(System.out::println);
```

```
// Verbose equivalent — avoid  
staff.forEach(new Consumer<Employee>() {  
    public void accept(Employee e) { System.out.println(e); }  
});
```

removeIf with Predicate

```
staff.removeIf(e -> e.getName().equals("An"));
```

```
staff.removeIf(e -> e instanceof PartTimeEmployee);
```

```
staff.removeIf(e -> e.pay() < 500);
```

```
// Predicate true → element is removed
```

```
// Safer than for-each + staff.remove
```

sort with Comparator Lambda

```
staff.sort((a, b) -> a.getName().compareTo(b.getName()));
```

```
// Preferred style
```

```
staff.sort(Comparator.comparing(Employee::getName));
```

```
staff.sort(Comparator.comparingDouble(Employee::pay).reversed());
```

```
staff.sort(Comparator.comparing(Employee::getName)  
            .thenComparing(Employee::getEmployeeId));
```

When Lambdas Shine

Short behavior

One condition or one action — keep it concise

Library hooks

`forEach`, `removeIf`, `sort` expect functional types

Avoid anonymous classes

Less boilerplate than `new Comparator() { ... }`

Named methods still win

Complex logic → method, then method reference

5 · ITERATOR

Safe traversal and removal

Why Iterator · hasNext / next / remove

Why Iterator?

Not every collection has `get(i)`. Iterator is the uniform way to walk elements — and the safe way to remove while traversing.

Uniform access

List, Set, Queue — same `hasNext` / `next` pattern

No index required

Works where random access does not exist

Safe remove

`iterator.remove()` avoids `ConcurrentModificationException`

Behind for-each

Enhanced for uses Iterator under the hood

The Problem for-each Cannot Solve

```
// FAILS at runtime
for (Employee e : staff) {
    if (e instanceof PartTimeEmployee)
        staff.remove(e); // ConcurrentModificationException
}
```

```
// forEach has the same restriction
staff.forEach(e -> { if (...) staff.remove(e); }); // CME
```

```
// Iterator.remove() is the classic safe answer
```

Iterator Interface — Core API

hasNext()

true if another element remains

next()

returns the next element · advances the cursor

remove()

removes the last element returned by next()

Basic Traversal

```
Iterator<Employee> it = staff.iterator();  
while (it.hasNext()) {  
    Employee e = it.next();  
    System.out.println(e.getName() + " → " + e.pay());  
}
```

```
// Equivalent idea to: for (Employee e : staff)
```

Iterator Rules

`next()` before `remove()`

`remove` acts on the element just returned

One `remove` per `next`

Second `remove` without `next` → `IllegalStateException`

Do not mix with `staff.remove`

Structural change via the list still risks CME

Fail-fast iterators

`ArrayList` detects concurrent modification and throws

Iterator.remove vs removeIf

Iterator

- Explicit control
- Can decide mid-loop
- Works on older code
- More boilerplate

```
it.next();
```

```
if (cond) it.remove();
```

removeIf

- One-liner predicate
- Preferred when bulk
- Java 8+
- Clear intent

```
staff.removeIf(
```

```
    e -> e instanceof
```

```
        PartTimeEmployee);
```

Iterator Operations

Traverse

`hasNext + next` — visit every element

Find while walking

Stop early when a condition matches

Remove selected

`it.remove()` after `next()` — one or many

Copy / collect

Build another list during iteration

Remove One by employeeId

```
boolean removeById(String employeeId) {  
    Iterator<Employee> it = staff.iterator();  
    while (it.hasNext()) {  
        if (it.next().getEmployeeId().equals(employeeId)) {  
            it.remove();  
            usedIds.remove(employeeId); // keep Set in sync  
            return true;  
        }  
    }  
    return false;  
}
```

Remove Many by Criteria

```
int removePartTime() {  
    int n = 0;  
    Iterator<Employee> it = staff.iterator();  
    while (it.hasNext()) {  
        Employee e = it.next();  
        if (e instanceof PartTimeEmployee) {  
            usedIds.remove(e.getEmployeeId());  
            it.remove();  
            n++;  
        }  
    }  
    return n;  
}
```

Collect During Iteration

```
List<Employee> highEarners(double min) {  
    List<Employee> result = new ArrayList<>();  
    Iterator<Employee> it = staff.iterator();  
    while (it.hasNext()) {  
        Employee e = it.next();  
        if (e.pay() >= min) result.add(e);  
    }  
    return result;  
}
```

// Does not modify staff — read-only walk

Common Mistakes

```
// 1) remove before next
```

```
it.remove(); // IllegalStateException
```

```
// 2) remove twice without next
```

```
it.next(); it.remove(); it.remove(); // IllegalStateException
```

```
// 3) structural change via the list
```

```
while (it.hasNext()) {
```

```
    it.next();
```

```
    staff.remove(0); // ConcurrentModificationException
```

```
}
```

forEachRemaining (Java 8+)

```
Iterator<Employee> it = staff.iterator();  
// ... maybe skip some with next() ...  
it.forEachRemaining(e ->  
    System.out.println(e.getName()));  
  
// Processes all remaining elements via Consumer  
// After this, hasNext() is false
```

Removing Elements with Iterator

`iterator.remove()` is the supported way to delete the current element during a walk. It keeps the iterator and the collection consistent.

Contract

`remove()` deletes the element last returned by `next()`

Safe

No `ConcurrentModificationException` when used correctly

Sync domain

Also update `usedIds` when you remove an `Employee`

The Safe Pattern

```
Iterator<Employee> it = staff.iterator();
while (it.hasNext()) {
    Employee e = it.next(); // 1) advance
    if (shouldRemove(e)) {
        usedIds.remove(e.getEmployeeId());
        it.remove();        // 2) remove current
    }
}

// shouldRemove = your condition (type, pay, name, ...)
```

What Not to Do

```
// A) for-each + collection.remove → CME  
for (Employee e : staff) {  
    if (e.pay() < 500) staff.remove(e); // BAD  
}
```

```
// B) staff.remove during Iterator walk → CME  
while (it.hasNext()) {  
    it.next();  
    staff.remove(0); // BAD  
}
```

```
// C) it.remove() without next() → IllegalStateException
```

Iterator.remove vs removeIf — Choose

Situation

Prefer

Simple predicate, bulk delete

`removeIf(e -> ...)`

Need side effects per removal

Iterator (log, update Set)

Stop after first match

Iterator + return

Teaching / older Java

Iterator

Production one-liner

`removeIf`

removeIf + Keep usedIds in Sync

```
// Option A: Iterator (easy to sync Set)
```

```
while (it.hasNext()) {  
    Employee e = it.next();  
    if (e.pay() < 500) {  
        usedIds.remove(e.getEmployeeId());  
        it.remove();  
    }  
}
```

```
// Option B: removeIf then rebuild usedIds
```

```
staff.removeIf(e -> e.pay() < 500);  
usedIds.clear();  
for (Employee e : staff) usedIds.add(e.getEmployeeId());
```

6 · SET

Unique elements · no duplicates

Interface · characteristics · HashSet

What Is a Set?

Set is a Collection that contains no duplicate elements. Models the mathematical set: membership matters more than position.

Extends Collection

add, remove, contains, size, iterator, ...

No `get(i)`

Not a List — no positional access

add returns boolean

false if the element was already present

Equality

Duplicates decided by equals (and hashCode for HashSet)

Core Set Operations

```
Set<String> usedIds = new HashSet<>();
```

```
usedIds.add("E01");    // true — inserted  
usedIds.add("E02");    // true  
usedIds.add("E01");    // false — already there
```

```
usedIds.contains("E02"); // true  
usedIds.remove("E01");  // true if removed  
usedIds.size();         // unique count
```

Set Characteristics

No duplicates

At most one of each equal element

No index

Cannot `staff.get(0)` — use `contains` / `iteration`

Order?

`HashSet`: no guaranteed order · `LinkedHashSet`: insertion · `TreeSet`: sorted

Null

`HashSet` allows one null · `TreeSet` does not (natural order)

Fast membership

`HashSet` `contains`/`add`/`remove` average $O(1)$

Set vs List

List

Set

Duplicates

Allowed

Forbidden

Order

Insertion / index

Depends on impl

Access

get(i)

No index

Typical use

Roster, sequence

Unique ids, tags

Our domain

staff

usedIds

HashSet — Default Set

HashSet is backed by a hash table (actually a HashMap). Best average-case cost for add, remove, and contains.

Average $O(1)$

add · remove · contains

No order guarantee

Do not rely on iteration order

hashCode + equals

Must be consistent for element type

Not synchronized

Single-threaded by default

Demo — usedIds with HashSet

```
List<Employee> staff = new ArrayList<>();  
Set<String> usedIds = new HashSet<>();  
  
void addEmployee(Employee e) {  
    if (!usedIds.add(e.getEmployeeId()))  
        throw new IllegalStateException("duplicate id");  
    staff.add(e);  
}  
  
// add returns false → id already in the set
```

HashSet of Employee Objects

If you store `Employee` in a `Set`, override `equals` and `hashCode` (usually on `employeeId`). Otherwise two equal-looking employees may both enter the set.

```
// Safer for our course: Set of ids, List of employees
Set<String> usedIds;      // uniqueness
List<Employee> staff;     // order + objects

// Set<Employee> needs proper equals/hashCode on Employee
```

When to Use HashSet

Unique membership

Ids, emails, tags, visited nodes

Fast contains

"Have we seen this key?" in $O(1)$ average

Deduplicate

`new HashSet<>(list)` drops duplicates

Not for ordered roster

Use List when position and sequence matter

LinkedHashSet — Unique + Insertion Order

HashSet plus a linked list of entries. Still no duplicates, but iteration follows insertion order.

Unique elements

Same set contract as HashSet

Predictable iteration

Order = order of first successful add

Slightly more memory

Extra links between entries

Still $\sim O(1)$

add / contains / remove average constant time

LinkedHashSet Demo

```
Set<String> ids = new LinkedHashSet<>();  
ids.add("E03");  
ids.add("E01");  
ids.add("E02");  
ids.add("E01"); // ignored  
  
for (String id : ids)  
    System.out.print(id + " ");  
// E03 E01 E02 — insertion order
```

When to Use LinkedHashSet

Unique + stable order

First-seen order of employee ids, URLs, tokens

Deduplicate a list, keep order

```
new LinkedHashSet<>(list)
```

Not sorted

If you need sorted order → TreeSet

vs HashSet

Pay a little memory/CPU for predictable iteration

TreeSet — Unique + Sorted

A NavigableSet backed by a red-black tree. Elements are always ordered — by natural order or a Comparator.

Sorted iteration

Ascending by default

$O(\log n)$

add · remove · contains

Comparable / Comparator

Elements must be comparable

No null (typical)

Natural ordering rejects null

TreeSet Demo — Sorted Ids

```
Set<String> ids = new TreeSet<>();
```

```
ids.add("E03");
```

```
ids.add("E01");
```

```
ids.add("E02");
```

```
for (String id : ids)
```

```
    System.out.print(id + " ");
```

```
// E01 E02 E03 — sorted order
```

NavigableSet ops: first(), last(), headSet(), tailSet()

TreeSet with Comparator

```
// Descending ids
Set<String> ids = new TreeSet<>(Comparator.reverseOrder());

// TreeSet of Employee by name — needs Comparator
Set<Employee> byName = new TreeSet<>(
    Comparator.comparing(Employee::getName)
        .thenComparing(Employee::getEmployeeId));

// Consistency: compareTo>equals should agree for Set semantics
```

HashSet vs LinkedHashSet vs TreeSet

	HashSet	LinkedHashSet	TreeSet
Order	None	Insertion	Sorted
add/contains	$O(1)^*$	$O(1)^*$	$O(\log n)$
Null	One OK	One OK	Usually no
Use when	Fast unique	Unique + order	Unique + sort

Choosing a Set Implementation

Need

Choose

Fast unique membership only

HashSet

Unique + remember insertion order

LinkedHashSet

Unique + always sorted

TreeSet

employeeId uniqueness next to List

HashSet usedIds

Sorted unique Employee by name

TreeSet + Comparator

Employee — equals & hashCode

```
// Required for HashSet / LinkedHashSet of Employee
```

```
@Override public boolean equals(Object o) {
```

```
    if (this == o) return true;
```

```
    if (!(o instanceof Employee e)) return false;
```

```
    return employeeId.equals(e.employeeId);
```

```
}
```

```
@Override public int hashCode() {
```

```
    return employeeId.hashCode();
```

```
}
```

```
// Uniqueness = same employeeId
```

HashSet<Employee> — Unique by Id

```
Set<Employee> unique = new HashSet<>();  
unique.add(new FullTimeEmployee("E01", "An", 1500));  
unique.add(new PartTimeEmployee("E02", "Binh", 10, 80));  
unique.add(new FullTimeEmployee("E01", "An Other", 2000));  
// add returns false — same employeeId, not inserted  
  
unique.size(); // 2  
for (Employee e : unique) System.out.println(e);  
// iteration order NOT guaranteed
```

LinkedHashSet<Employee> — Keep Order

```
Set<Employee> ordered = new LinkedHashSet<>();  
ordered.add(new FullTimeEmployee("E03", "Chi", 1800));  
ordered.add(new FullTimeEmployee("E01", "An", 1500));  
ordered.add(new PartTimeEmployee("E02", "Binh", 10, 80));  
  
for (Employee e : ordered)  
    System.out.println(e.getEmployeeId());  
// E03, E01, E02 — first-insert order
```

TreeSet<Employee> — Sorted by Name

```
Set<Employee> sorted = new TreeSet<>(
    Comparator.comparing(Employee::getName)
        .thenComparing(Employee::getEmployeeId));

sorted.add(new FullTimeEmployee("E03", "Chi", 1800));
sorted.add(new FullTimeEmployee("E01", "An", 1500));
sorted.add(new PartTimeEmployee("E02", "Binh", 10, 80));

for (Employee e : sorted)
    System.out.println(e.getName());
// An, Binh, Chi — sorted
```

Same Adds — Different Iteration

Add order: E03, E01, E02

HashSet → ??? (any order)

LinkedHashSet → E03, E01, E02

TreeSet(name) → An, Binh, Chi (by name)

All three still reject a second E01.

Recommendation for Our Domain

Default teaching model

List<Employee> staff + Set<String> usedIds (HashSet)

Set<Employee>

OK after equals/hashCode — or TreeSet with Comparator

Need first-seen order of ids

LinkedHashSet<String>

Need sorted unique names/ids

TreeSet

7 · MAP

Key → Value associations

Interface · characteristics · HashMap

What Is a Map?

Map stores key-value pairs. Keys are unique; each key maps to at most one value. Map is not a Collection — parallel hierarchy.

put(K, V)

Insert or replace the value for a key

get(K)

Value for key, or null if absent

containsKey / containsValue

Membership tests

remove(K)

Delete the entry for that key

keySet / values / entrySet

Views to iterate keys, values, or pairs

Core Map Operations

```
Map<String, Employee> byId = new HashMap<>();
```

```
byId.put("E01", an);
```

```
byId.put("E02", binh);
```

```
Employee e = byId.get("E01"); // an or null
```

```
byId.containsKey("E03");    // false
```

```
byId.remove("E02");
```

```
byId.size();                // number of keys
```

Map Characteristics

Unique keys

put same key again → replaces the value

Values may duplicate

Two keys can map to equal values

Not a Collection

Cannot write `Collection c = new HashMap<>()`

One null key?

HashMap allows one null key · TreeMap does not

Lookup by key

Natural fit for `employeeId` → `Employee`

Map vs List vs Set

	List	Set	Map
Stores	Elements	Elements	Key→Value
Access	Index	Membership	By key
Duplicates	OK	No	Keys unique
Our use	staff	usedIds	byId

HashMap — Default Map

Hash table implementation of Map. Best average-case performance for put, get, and remove by key.

Average $O(1)$

put · get · remove · containsKey

No order guarantee

Do not rely on iteration order

Null key / values

One null key · null values allowed

Not synchronized

Single-threaded by default

Demo — byId Lookup

```
Map<String, Employee> byId = new HashMap<>();

void addEmployee(Employee e) {
    if (byId.containsKey(e.getEmployeeId()))
        throw new IllegalStateException("duplicate id");
    staff.add(e);
    byId.put(e.getEmployeeId(), e); // O(1) index
}

Employee e = byId.get("E02"); // no list scan
```

Iterating a Map

```
// Keys
```

```
for (String id : byId.keySet())
```

```
    System.out.println(id);
```

```
// Values
```

```
for (Employee e : byId.values())
```

```
    System.out.println(e.pay());
```

```
// Entries (preferred when you need both)
```

```
for (Map.Entry<String, Employee> en : byId.entrySet())
```

```
    System.out.println(en.getKey() + " → " + en.getValue());
```

When to Use HashMap

Lookup by key

id → Employee, code → product, word → count

Replace linear findById

$O(n)$ list scan → $O(1)$ map get

Unique keys built-in

put overwrites — or check containsKey first

Not ordered

Need order → LinkedHashMap or TreeMap later

Key Takeaways — Map & HashMap

- 01 Map = unique keys → values · not a Collection
- 02 put / get / containsKey / remove — core API
- 03 HashMap: average $O(1)$ lookup — default choice
- 04 Domain: `Map<String, Employee>` `byId` upgrades `findById`
- 05 Iterate with `keySet`, `values`, or `entrySet`

LinkedHashMap — Map + Insertion Order

HashMap with a linked list of entries. Iteration follows insertion order (or access order if configured).

Unique keys

Same Map contract

Predictable iteration

keySet / entrySet walk in put order

~O(1)

put / get / remove average constant time

Access-order mode

Optional — basis for simple LRU caches

LinkedHashMap Demo

```
Map<String, Employee> byId = new LinkedHashMap<>();  
byId.put("E03", chi);  
byId.put("E01", an);  
byId.put("E02", binh);  
  
for (String id : byId.keySet())  
    System.out.print(id + " ");  
// E03 E01 E02 — insertion order
```

TreeMap — Map + Sorted Keys

A NavigableMap backed by a red-black tree. Keys are always ordered — natural order or a Comparator.

Sorted keys

Iteration follows key order

$O(\log n)$

put · get · remove

Navigable

firstKey, lastKey, headMap, tailMap, ...

No null key

With natural ordering

TreeMap Demo — Sorted employeeId

```
Map<String, Employee> byId = new TreeMap<>();
```

```
byId.put("E03", chi);
```

```
byId.put("E01", an);
```

```
byId.put("E02", binh);
```

```
for (String id : byId.keySet())
```

```
    System.out.print(id + " ");
```

```
// E01 E02 E03 — sorted keys
```

```
byId.firstKey(); // E01
```

```
byId.lastKey();  // E03
```

TreeMap with Comparator

```
// Descending keys
```

```
Map<String, Employee> byId =  
    new TreeMap<>(Comparator.reverseOrder());
```

```
// Keys must be mutually comparable
```

```
// Custom Comparator if key type has no natural order
```

HashMap vs LinkedHashMap vs TreeMap

	HashMap	LinkedHashMap	TreeMap
Key order	None	Insertion*	Sorted
put/get	$O(1)^*$	$O(1)^*$	$O(\log n)$
Null key	One OK	One OK	No (natural)
Use when	Fast lookup	Lookup + order	Lookup + sort

Choosing for Employee byId

HashMap

Default — fastest findById replacement

LinkedHashMap

Need to print employees in registration order via keys

TreeMap

Need keys always sorted (reports by id sequence)

With List staff

Map for lookup · List for sequence / payroll walk

Map CRUD Operations

Create

`put(key, value)` — insert new mapping

Read

`get(key)` · `containsKey` · `getOrDefault`

Update

`put` same key again · `replace` · `compute`

Delete

`remove(key)` · `clear()`

CRUD on Map<String, Employee>

```
Map<String, Employee> byId = new HashMap<>();
```

```
// Create
```

```
byId.put("E01", new FullTimeEmployee("E01", "An", 1500));
```

```
// Read
```

```
Employee e = byId.get("E01");
```

```
Employee x = byId.getDefault("E99", null);
```

```
// Update (same key)
```

```
byId.put("E01", new FullTimeEmployee("E01", "An", 1600));
```

```
// Delete
```

```
byId.remove("E01");
```

Create — Reject Duplicate Key

```
void putNew(Employee e) {  
    String id = e.getEmployeeId();  
    if (byId.containsKey(id))  
        throw new IllegalStateException("duplicate id");  
    byId.put(id, e);  
}  
  
// Or: byId.putIfAbsent(id, e) returns previous or null
```

keySet() — View of Keys

```
Set<String> ids = byId.keySet();
```

```
for (String id : ids)  
    System.out.println(id);
```

```
ids.contains("E01"); // true if mapped
```

```
// Live view: removing from keySet removes the entry  
ids.remove("E02"); // also removes from byId
```

```
// Cannot add via keySet — keys need values
```

values() — View of Values

```
Collection<Employee> all = byId.values();
```

```
double total = 0;
```

```
for (Employee e : all)
```

```
    total += e.pay(); // polymorphic
```

```
// values() is a Collection, not a List — no get(i)
```

```
// Removing a value removes one matching entry
```

entrySet() — Key and Value Together

```
for (Map.Entry<String, Employee> en : byId.entrySet()) {  
    String id = en.getKey();  
    Employee e = en.getValue();  
    System.out.println(id + " → " + e.getName() + " / " + e.pay());  
}
```

// Preferred when you need both key and value

// en.setValue(newEmp) updates the map in place

Setup — Map as Primary Store

```
Map<String, Employee> byId = new HashMap<>();
```

```
byId.put("E01", new FullTimeEmployee("E01", "An", 1500));
```

```
byId.put("E02", new PartTimeEmployee("E02", "Binh", 10, 80));
```

```
byId.put("E03", new FullTimeEmployee("E03", "Chi", 1800));
```

```
byId.size(); // 3
```

Employee Lookup by ID

```
Employee findById(String employeeId) {  
    return byId.get(employeeId); // O(1)  
}
```

```
Employee e = findById("E02");  
if (e != null) {  
    System.out.println(e.getName() + " pays " + e.pay());  
}
```

```
// vs List scan: for (Employee e : staff) if id equals ...
```

Traversing Map Entries

```
// 1) entrySet — both sides
```

```
byId.forEach((id, e) ->
```

```
    System.out.println(id + " → " + e.getName()));
```

```
// 2) Classic entry loop
```

```
for (var en : byId.entrySet())
```

```
    System.out.println(en.getKey() + ": " + en.getValue().pay());
```

```
// 3) Only values — total payroll
```

```
double sum = byId.values().stream()
```

```
    .mapToDouble(Employee::pay).sum();
```

8 · QUEUE AND DEQUE

FIFO processing · double-ended queues

Queue · PriorityQueue · Deque · ArrayDeque

What Is a Queue?

Queue is a Collection designed for holding elements prior to processing — typically FIFO: first in, first out.

Offer / add

Insert at the tail

Poll / remove

Take from the head

Peek / element

Look at the head without removing

Two styles

Throwing methods vs special-value methods

Queue Operations — Two Families

Action**Throws exception****Special value****Insert**`add(e)``offer(e) → false if full`**Remove head**`remove()``poll() → null if empty`**Examine head**`element()``peek() → null if empty`

Queue Usage Pattern

```
Queue<Employee> waiting = new ArrayDeque<>();
```

```
waiting.offer(an);
```

```
waiting.offer(binh);
```

```
Employee next = waiting.poll(); // an — FIFO
```

```
Employee look = waiting.peek(); // binh — still in queue
```

```
while ((next = waiting.poll()) != null) {
```

```
    process(next);
```

```
}
```

PriorityQueue — Ordered by Priority

A Queue where the head is the least element under a Comparator (or natural order). Not FIFO — highest priority leaves first.

Heap under the hood

offer / poll $O(\log n)$ · peek $O(1)$

No null elements

PriorityQueue rejects null

Unordered iteration

for-each does not walk in priority order

Comparator

Define what “higher priority” means

PriorityQueue by pay()

```
// Highest pay processed first
Queue<Employee> pq = new PriorityQueue<>(
    Comparator.comparingDouble(Employee::pay).reversed());

pq.offer(an); // 1500
pq.offer(chi); // 1800
pq.offer(binh); // 800

pq.poll(); // chi (1800)
pq.poll(); // an (1500)
```

Deque — Double-Ended Queue

Deque extends Queue and allows insert/remove at both ends. Can act as a queue (FIFO) or a stack (LIFO).

First end

`addFirst / offerFirst · removeFirst / pollFirst · peekFirst`

Last end

`addLast / offerLast · removeLast / pollLast · peekLast`

As stack

`push = addFirst · pop = removeFirst`

As queue

`offerLast · pollFirst` — standard FIFO

ArrayDeque — Fast General-Purpose Deque

Resizable-array implementation of Deque. Preferred over Stack and often over LinkedList for queue/stack use.

No capacity limit (grows)

Like ArrayList growth

No null elements

Null not allowed

Not thread-safe

Single-threaded default

Default Queue/Deque choice

```
new ArrayDeque<>()
```

Queue vs Deque

	Queue	Deque
Ends used	One (head/tail FIFO)	Both ends
Stack use	No	Yes (push/pop)
Typical impl	ArrayDeque / PriorityQueue	ArrayDeque
Priority order	PriorityQueue	Not by priority

Employee Processing Queue (FIFO)

```
Queue<Employee> desk = new ArrayDeque<>();  
  
desk.offer(an);  
desk.offer(binh);  
desk.offer(chi);  
  
while (!desk.isEmpty()) {  
    Employee e = desk.poll();  
    System.out.println("Processing " + e.getName());  
    // print payslip, approve leave, ...  
}  
  
// Order: An → Binh → Chi
```

Priority-based Employee Processing

```
Queue<Employee> urgent = new PriorityQueue<>(
    Comparator.comparingDouble(Employee::pay).reversed());

urgent.offer(binh); // low pay
urgent.offer(chi); // high pay
urgent.offer(an);

while (!urgent.isEmpty()) {
    Employee e = urgent.poll();
    System.out.println(e.getName() + " pay=" + e.pay());
}

// Chi first, then An, then Binh
```

Key Takeaways — Collections Framework

- 01 Collections replace hand-built dynamic arrays with standard APIs
- 02 Program to interfaces: List, Set, Queue, Map — pick by need
- 03 List: ordered + index · Set: unique · Map: key→value · Queue: process order
- 04 ArrayList / HashSet / HashMap are the everyday defaults
- 05 Iterator / removeIf for safe removal · Comparator for sort and Tree* types
- 06 Domain pattern: List staff + Set usedIds or Map byId for O(1) lookup